License Plate Deblurring

INTRODUCTION TO COMPUTER VISION — BIG DATA & AI

Students : Aya EL MOUDEN (ID: 120020) · Adam HADOUTI (ID: 118849)

Professor: Ilias TOUGUI

2025–2026

ESIN — École Supérieure d'Informatique et du Numérique, UIR
[Company address]

Abstract. Motion blur from fast-moving vehicles makes license plate recognition challenging. This project develops a pipeline that estimates the motion blur kernel (PSF) from the degraded image and recovers the sharp plate using Wiener filter deconvolution. Working on 11 real Kaggle plates with synthetic blur, the pipeline achieves a mean PSNR gain of **+7.74 dB** on the validation set, outperforming standard sharpening baselines by over 8 dB. An interactive GUI and an optional CNN-style blur estimator are also implemented.

1. Introduction

Automatic License Plate Recognition (ALPR) is used in traffic surveillance, toll collection, and parking management. When vehicles move at high speed, the camera smears the plate characters into an unreadable streak. Reversing this degradation is a classical ill-posed inverse problem requiring modelling of the blur process and its inversion.

This project applies course concepts directly — 2-D DFT, the convolution theorem, linear filtering, and Wiener filtering — to solve a real-world problem. All code is implemented in Python (NumPy, OpenCV, scikit-image) in a single runnable Jupyter notebook.

2. Dataset

We use the Large License Plate Dataset (Kaggle, Fares El Menshawi, 2023) containing 25,470 annotated vehicle photos in YOLO format. A custom script (crop_plates.py) extracts plate regions using bounding box labels, resizes to 320x100 px, and filters by Laplacian variance (threshold = 50). 11 sharp plates were retained covering US, German, Spanish and South African formats, split 70/30 into training (7) and validation (4) sets.

Property	Value
Plates retained	11 — 320x100 px grayscale JPEG
Train / Val split	7 training | 4 validation (70/30)
Blur configs	5 configs: L = 15–35 px, Angle = 0°–45°
Noise	AWGN sigma = 5.0

3. Methodology

3.1 Degradation model

The blurred image g is modelled as the convolution of the sharp image f with a motion PSF h, plus AWGN noise n:

```
g(x,y) = h(x,y) * f(x,y) + n(x,y)
```

By the convolution theorem this becomes pointwise multiplication in the frequency domain: $G = H \cdot F + N$, enabling O(N log N) processing via FFT.

3.2 PSF construction

A motion PSF of length L at angle θ traces unit impulses through the kernel centre and normalises to sum = 1. Five configs used: L = 15, 20, 25, 30, 35 px at angles 0°, 0°, 15°, 30°, 45°.

3.3 Blur estimation via FFT

Motion blur creates periodic dark fringes (nulls) in the FFT magnitude spectrum, perpendicular to the blur direction. For each candidate (L, A), we score the alignment of PSF nulls with low-energy regions of the observed

spectrum, returning the best-scoring pair. Achieves ~70% length accuracy (±5 px) and ~60% angle accuracy (±15°).

3.4 Wiener filter deconvolution

The optimal linear MMSE estimate in the frequency domain:

```
F_hat(u,v) = [ H*(u,v) / ( |H(u,v)|² + K ) ] × G(u,v)
```

K is the regularisation parameter. Sweeping K ∈ {10^{-4} … 0.1}, we select **K = 0.01** as the best trade-off between noise suppression and sharpness recovery (PSNR = 19.6 dB vs 9.3 dB at K = 10^{-4}).

3.5 Optional CNN-style estimator

A Random Forest (300 trees) trained on 64-bin radially-averaged FFT power spectrum features achieves 100% training accuracy and **75% validation accuracy** on blur config classification — demonstrating the CNN concept without GPU requirements.

4. Results

4.1 PSNR and SSIM

Metric	Blurred	Restored	Gain
PSNR — validation (dB)	11.02	18.76	+7.74
SSIM — validation	0.084	0.478	+0.394
PSNR — training (dB)	11.32	20.27	+8.96
SSIM — training	0.070	0.556	+0.486

4.2 Baseline comparison

Method	PSNR (dB)	Std	Note
Blurred input	11.02	2.36	Reference
Gaussian Sharpen	10.69	2.08	Worse — amplifies noise
Unsharp Mask	10.24	1.76	Worse — edge ringing
Wiener Filter	**18.76**	2.15	**Best — physics-based**

Gaussian Sharpen and Unsharp Mask perform worse than the blurred input on some plates because they amplify noise without inverting the blur kernel. The Wiener filter achieves a consistent **+7 to +9 dB** improvement across all plates.

4.3 Real-world extension

Realistic blur (L = 20–25 px) applied to 4 real plates. Wiener wins on every plate:

Plate	Blurry	Wiener	Gauss	Unsharp
6NCE831	10.0	18.7	9.6	9.1
N·HZ1002	11.0	19.0	10.6	9.9
815-VWV	13.0	17.5	12.6	12.1
0695 DVH	13.0	17.6	12.7	12.4

5. Challenges

- **PSF 180° symmetry.** FFT magnitude is centro-symmetric — blur at θ is indistinguishable from θ+180°. Resolved by restricting search to [0°, 180°).
- **Regularisation K.** Small K amplifies noise (Gibbs ringing); large K leaves blur. K = 0.01 selected by 5-value sweep.
- **Gibbs artefacts.** Ringing at PSF null frequencies — directly related to the Gibbs effect from course lectures. Mitigated by K regularisation.
- **Disk space.** Only 160 MB free. Custom crop script reduces storage by >95% by extracting only plate crops.
- **Path portability.** Three-level path resolution ensures the notebook runs on any machine without manual configuration.
- **Diagonal blur.** Accuracy drops to ~60% for diagonal PSFs because closely-spaced nulls overlap with image texture.

6. Course Concept Connections

Concept	How it is used in this project
2-D DFT / FFT	Forward blur model (G=H·F) and deconvolution backbone.
Convolution theorem	g=h*f in space ↔ G=H·F in frequency — O(N log N).
Wiener filter	$\hat{F}=H^*G/(\|H\|^2+K)$ — optimal MMSE estimator.
PSF / Impulse response	Motion PSF is the camera-vehicle system impulse response.
PSNR & SSIM	Quality metrics evaluated on all train/val experiments.
Gibbs effect	Ringing at PSF nulls — mitigated by K regularisation.
AWGN noise model	Applied during blur generation; K = noise/signal ratio.
Linear filtering LSI	Motion blur is an LSI system inverted in frequency domain.

7. New Concepts Explored

Concept	Description
Blind deconvolution	PSF estimation from blurred image without prior knowledge.
ipywidgets	Interactive sliders for real-time deblurring GUI in Jupyter.
YOLO bbox format	Parsed to extract plate crops from annotated vehicle images.
Laplacian variance	Fast parameter-free sharpness score for dataset filtering.
RF on FFT features	CNN-style blur estimator without GPU — 75% val accuracy.

8. References

[1] Gonzalez & Woods. Digital Image Processing, 4th ed., Pearson, 2018 — Ch. 5.
[2] Jain, A.K. Fundamentals of Digital Image Processing, Prentice Hall — Ch. 7.
[3] El Menshawi, F. Large License Plate Dataset, Kaggle, 2023.
[4] OpenCV. Fourier Transform Tutorial — docs.opencv.org.
[5] scikit-image. PSNR & SSIM — scikit-image.org/docs/stable/api/skimage.metrics.html.
[6] Tougui, I. Course Slides: Wiener Filtering & PSF Estimation — ESIN UIR, 2024–2025.
[7] Levin et al. Understanding and Evaluating Blind Deconvolution Algorithms, CVPR 2009.
[8] ipywidgets Team. Interactive HTML Widgets for Jupyter — ipywidgets.readthedocs.io.

Key Results: Validation PSNR +7.74 dB · Wiener beats all baselines by >8 dB · 75% CNN estimator accuracy · No GPU required